

NN291125

November 29, 2025

```
[1]: # Paso 1. Librerías y carga de datos

from sklearn.model_selection import train_test_split # Para separar los datos
    ↪ en entrenamiento y prueba
from sklearn.preprocessing import StandardScaler # Para escalar los datos
from sklearn.datasets import load_breast_cancer # Para los datos que voy a usar
from sklearn.neural_network import MLPClassifier # Librería Red Neuronal
from sklearn.metrics import classification_report, confusion_matrix # para
    ↪ evaluación
```

```
[2]: cancer = load_breast_cancer()
```

```
[3]: cancer.keys()
```

```
[3]: dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names',
               'filename', 'data_module'])
```

```
[4]: # Paso 2. Análisis exploratorio de mis datos
print(cancer['DESCR'])
```

```
.. _breast_cancer_dataset:
```

```
Breast cancer wisconsin (diagnostic) dataset
-----
```

```
**Data Set Characteristics:**
```

```
:Number of Instances: 569
```

```
:Number of Attributes: 30 numeric, predictive attributes and the class
```

```
:Attribute Information:
```

- radius (mean of distances from center to points on the perimeter)
- texture (standard deviation of gray-scale values)
- perimeter
- area
- smoothness (local variation in radius lengths)
- compactness (perimeter² / area - 1.0)

- concavity (severity of concave portions of the contour)
- concave points (number of concave portions of the contour)
- symmetry
- fractal dimension ("coastline approximation" - 1)

The mean, standard error, and "worst" or largest (mean of the three worst/largest values) of these features were computed for each image, resulting in 30 features. For instance, field 0 is Mean Radius, field 10 is Radius SE, field 20 is Worst Radius.

- class:
 - WDBC-Malignant
 - WDBC-Benign

:Summary Statistics:

	Min	Max
radius (mean):	6.981	28.11
texture (mean):	9.71	39.28
perimeter (mean):	43.79	188.5
area (mean):	143.5	2501.0
smoothness (mean):	0.053	0.163
compactness (mean):	0.019	0.345
concavity (mean):	0.0	0.427
concave points (mean):	0.0	0.201
symmetry (mean):	0.106	0.304
fractal dimension (mean):	0.05	0.097
radius (standard error):	0.112	2.873
texture (standard error):	0.36	4.885
perimeter (standard error):	0.757	21.98
area (standard error):	6.802	542.2
smoothness (standard error):	0.002	0.031
compactness (standard error):	0.002	0.135
concavity (standard error):	0.0	0.396
concave points (standard error):	0.0	0.053
symmetry (standard error):	0.008	0.079
fractal dimension (standard error):	0.001	0.03
radius (worst):	7.93	36.04
texture (worst):	12.02	49.54
perimeter (worst):	50.41	251.2
area (worst):	185.2	4254.0
smoothness (worst):	0.071	0.223
compactness (worst):	0.027	1.058
concavity (worst):	0.0	1.252
concave points (worst):	0.0	0.291
symmetry (worst):	0.156	0.664

fractal dimension (worst): 0.055 0.208
=====

:Missing Attribute Values: None

:Class Distribution: 212 - Malignant, 357 - Benign

:Creator: Dr. William H. Wolberg, W. Nick Street, Olvi L. Mangasarian

:Donor: Nick Street

:Date: November, 1995

This is a copy of UCI ML Breast Cancer Wisconsin (Diagnostic) datasets.
<https://goo.gl/U2Uwz2>

Features are computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. They describe characteristics of the cell nuclei present in the image.

Separating plane described above was obtained using Multisurface Method-Tree (MSM-T) [K. P. Bennett, "Decision Tree Construction Via Linear Programming." Proceedings of the 4th Midwest Artificial Intelligence and Cognitive Science Society, pp. 97-101, 1992], a classification method which uses linear programming to construct a decision tree. Relevant features were selected using an exhaustive search in the space of 1-4 features and 1-3 separating planes.

The actual linear program used to obtain the separating plane in the 3-dimensional space is that described in: [K. P. Bennett and O. L. Mangasarian: "Robust Linear Programming Discrimination of Two Linearly Inseparable Sets", Optimization Methods and Software 1, 1992, 23-34].

This database is also available through the UW CS ftp server:

ftp ftp.cs.wisc.edu
cd math-prog/cpo-dataset/machine-learn/WDBC/

.. dropdown:: References

- W.N. Street, W.H. Wolberg and O.L. Mangasarian. Nuclear feature extraction for breast tumor diagnosis. IS&T/SPIE 1993 International Symposium on Electronic Imaging: Science and Technology, volume 1905, pages 861-870, San Jose, CA, 1993.
- O.L. Mangasarian, W.N. Street and W.H. Wolberg. Breast cancer diagnosis and prognosis via linear programming. Operations Research, 43(4), pages 570-577,

- W.H. Wolberg, W.N. Street, and O.L. Mangasarian. Machine learning techniques to diagnose breast cancer from fine-needle aspirates. Cancer Letters 77 (1994) 163-171.

[illegible]

```
X_train, X_test, y_train, y_test = train_test_split(X,y)
```

```
[28]: # Crear un escalador para normalizar los datos, ya que las redes neuronales son
      ↪ muy sensibles a las escalas
      scaler = StandardScaler()
```

```
[29]: # Usar matriz de entrenamiento para escalar
      scaler.fit(X_train)
```

```
[29]: StandardScaler()
```

```
[30]: #Aplicar la transformación (escalación) a mis matrices de entrenamiento y de
      ↪ prueba
      X_train = scaler.transform(X_train)
      X_test = scaler.transform(X_test)
```

```
[31]: # Crear el modelo vacio pero listo para entrenar
      mlp = MLPClassifier(hidden_layer_sizes = (30,30,30))
```

```
[32]: # Entrenar el modelo con los datos de entrenamiento
      mlp.fit(X_train, y_train)
```

```
[32]: MLPClassifier(hidden_layer_sizes=(30, 30, 30))
```

```
[33]: # Para ver como quedaron los parametros de mi red neuronal
      mlp.get_params()
```

```
[33]: {'activation': 'relu',
      'alpha': 0.0001,
      'batch_size': 'auto',
      'beta_1': 0.9,
      'beta_2': 0.999,
      'early_stopping': False,
      'epsilon': 1e-08,
      'hidden_layer_sizes': (30, 30, 30),
      'learning_rate': 'constant',
      'learning_rate_init': 0.001,
      'max_fun': 15000,
      'max_iter': 200,
      'momentum': 0.9,
      'n_iter_no_change': 10,
      'nesterovs_momentum': True,
      'power_t': 0.5,
      'random_state': None,
      'shuffle': True,
      'solver': 'adam',
      'tol': 0.0001,
      'validation_fraction': 0.1,
      'verbose': False,
```

```
'warm_start': False}
```

```
[34]: # Paso 4. Predicciones
```

```
predictions = mlp.predict(X_test)
```

```
[35]: # Paso 5. Evaluación
```

```
print(confusion_matrix(y_test, predictions))
```

```
[[55  4]
 [ 2 82]]
```

```
[36]: print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
0	0.96	0.93	0.95	59
1	0.95	0.98	0.96	84
accuracy			0.96	143
macro avg	0.96	0.95	0.96	143
weighted avg	0.96	0.96	0.96	143

```
[37]: len(mlp.coefs_)
```

```
[37]: 4
```

```
[38]: len(mlp.coefs_[0])
```

```
[38]: 30
```

```
[39]: mlp.coefs_[0][25]
```

```
[39]: array([-0.02299851,  0.04469212, -0.18980353,  0.0135556 , -0.24811889,
        -0.20891588, -0.22714664,  0.1491057 ,  0.31450458, -0.04736088,
         0.14428145,  0.22029506,  0.19526568,  0.24579644,  0.05632641,
        -0.24365352, -0.21266861,  0.02788552,  0.00438352,  0.31153352,
        -0.2343592 ,  0.22793462,  0.13298363, -0.17583582, -0.1493404 ,
        -0.2175636 , -0.23471128,  0.00742895,  0.12636099, -0.06791563])
```

```
[40]: len(mlp.intercepts_)
```

```
[40]: 4
```

```
[42]: len(mlp.intercepts_[0])
```

```
[42]: 30
```

```
[44]: mlp.intercepts_[0][25]
```

```
[44]: 0.023240715186020076
```

```
[ ]:
```